

Передача параметров конструкторам базовых классов

A ctor(x)
↑
B ctor(x,y)

```
ctor-производного(params):  
    ctor_БАЗОВОГО_1(params 1),  
    ctor_БАЗОВОГО_2(params 2),  
    ...  
    ctor_БАЗОВОГО_N(params N),  
    поле 1  
    ...  
    поле M } НАШИ поля  
    {  
        Тело_ctor  
    }
```

Мораль



Конструктор дочернего класса должен принимать параметры, чтобы передать их «транзитом» вверх суперклассам, ДАЖЕ ЕСЛИ У НЕГО НЕ ЗАДУМЫВАЛИСЬ СВОИ ПАРАМЕТРЫ



После двоеточия можно звать конструкторы суперклассов и заполнять свои поля



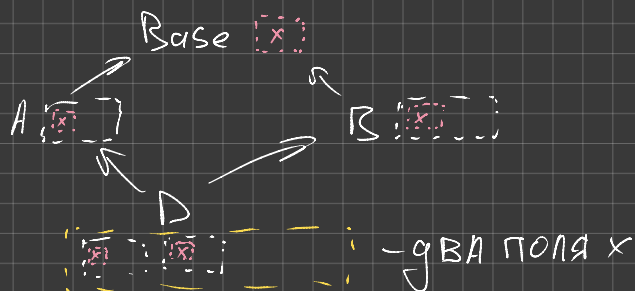
Сначала звать конструкторы суперклассов - в порядке наследования, потом заполнять поля - в порядке объявления

4) Полиморфизм

Виртуальное наследование

У нас проблема: Бриллиантовое наследование
это также краткий псевдокод

```
Base :  
    public:  
        int x;  
A : public Base;  
B : public Base;  
D : public A, public B;
```



И что?

D d;

d.x = 5; - ОШИБКА: НЕ ЗНАЕМ, КАКОЙ X НУЖЕН

РЕШЕНИЯ

1. Операция разрешения контекста ::

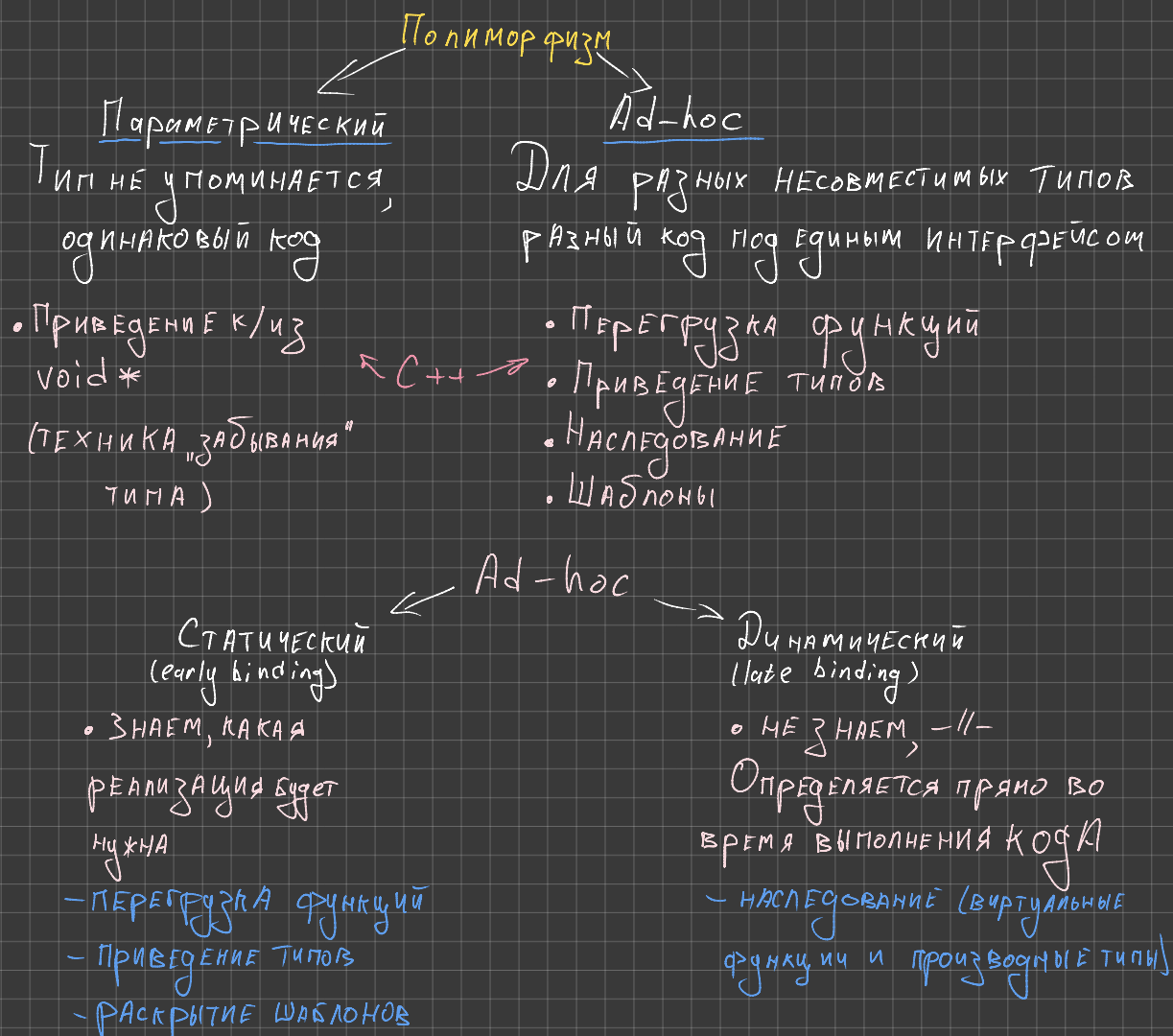
d.A::x = 5;
d.B::x = -30; | Всё ОК

2. Виртуальное наследование

A: virtual Base; | ГАРАНТИРУЕТСЯ, ЧТО БУДЕТ
B: virtual Base; | ровно одна копия Base в каждом наследнике

Теперь d.A::x и d.B::x - одно и то же

Полиморфизм — способность функции обрабатывать данные разных типов

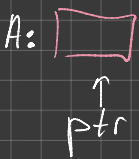


УКАЗАТЕЛИ НА ПРОИЗВОДНЫЕ ТИПЫ

УКАЗАТЕЛЬ НА БАЗОВЫЙ КЛАСС, МОЖЕТ УКАЗЫВАТЬ НА ЛЮБОГО НАСЛЕДНИКА

❗ ОБРАТНО НЕ РАБОТАЕТ

```
A;  
B: A;  
A* ptr;  
A a;  
B b;  
ptr = &a; | OK  
ptr = &b;
```



ptr (указывает только на наследующую часть)

КАК ПОЛУЧИТЬ ДОСТУП

((B*) ptr) -> func_in_B();

static_cast<B*>(ptr) -> func_in_B();

Виртуальные функции (virtual)

- МОЖЕТ БЫТЬ ПЕРЕОПРЕДЕЛЕНА В НАСЛЕДНИКАХ
- ПОМЕЧАЕТСЯ virtual, при ПЕРЕОПРЕДЕЛЕНИИ — НЕОБЯЗАТЕЛЬНО
 ↑
 В БАЗОВОМ
- КОГ ЗАВИСИТ ОТ ТИПА ОБЪЕКТА

Класс с virtual — ПОЛИМОРФНЫЙ (ТАКЖЕ И НАСЛЕДУЕМЫЕ)

ПЕРЕОПРЕДЕЛЕНИЕ / ПЕРЕГРУЗКА

- Virtual-функция, которая ПЕРЕОПРЕДЕЛЯЕТСЯ — ПЕРЕОПРЕДЕЛЕННАЯ

- НЕ СЛУЧАЙ ПЕРЕГРУЗКИ

ПЕРЕГРУЗКА -> РАЗНЫЕ ПАРАМЕТРЫ

ПЕРЕОПРЕДЕЛЕНИЕ -> ОДИНАКОВЫЕ ПАРАМЕТРЫ

РАЗНЫЕ ТИПЫ ОБЪЕКТОВ, ОТ КОТОРЫХ ВЫЗЫВАЕМСЯ

- Virtual func -> МЕТОД КЛАССА, НЕ friend (МБ friend для других классов)

Пример

who() - ВЫВОДИТ СВОЙ КЛАСС

Base:

virtual who(); ← КАКАЯ-ТО РЕАЛИЗАЦИЯ

D1: Base:

who();

D2: D1:

who();

→ ТИПЫ СООТВЕТСТВУЮТ

Base. who();

Out:

D1. who();

Base

D2. who();

D1

D2

changes:

D2: Base:

НЕТ СВОЕЙ РЕАЛИЗАЦИИ who()

Out:

Base

D1

Base

changes:

D2: D1:

-//-

Out:

Base

D1

D1

ПРАВИЛО: идём ВВЕРХ ПО СТРЕЛОЧКАМ ГРАФА НАСЛЕДОВАНИЯ, ПОКА НЕ НАЙДЁМ БЛИЖАЙШУЮ РЕАЛИЗАЦИЮ

Виртуальные деструкторы

- НЕТ virtual конструкторов
- ЕСТЬ virtual деструкторы
- ГАРАНТИРУЕТСЯ, ЧТО ПОЛИМОРФНЫЙ КЛАСС ПОЛУЧИТ ИМЕННО СВОЙ ДЕКТРУКТОР

Таблицы виртуальных функций

- хранит адреса virtual методов класса и его базовых классов
- vtable - таблица, vptr - указатель на vtable
- Base:

public:

virtual void foo();

virtual void bar();

virtual void baz();

I: Base:

public:

void bar();

virtual void qux();

Хранение в памяти:

	Base
	vptr
+2	Base::foo()
+3	Base::bar()
+4	Base::baz()

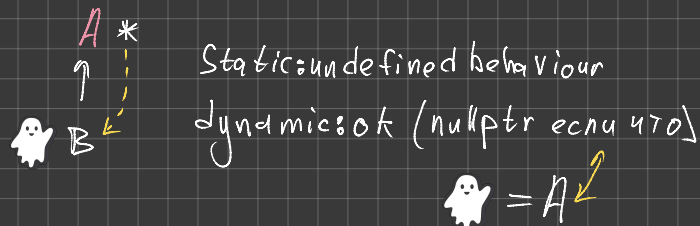
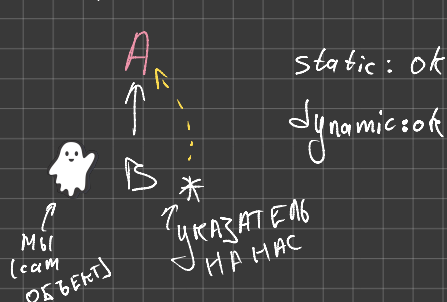
	I
	vptr
+2	Base::foo()
+3	I::bar()
+4	Base::baz()
+5	I::qux()

Dynamic_cast

- выполняет приведение типов, **делая проверку** в runtime
- всё плохо → вернёт nullptr или std::bad_cast
- только для **полиморфных** классов
- **нужно** делать при нисходящем или горизонтальном касте
- **не нужно** при восходящем → хватит и static_cast

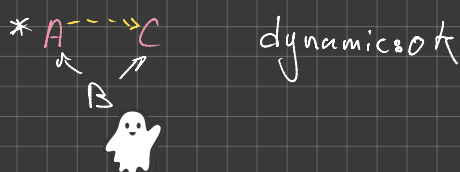
Upcast

Downcast:



ghost = A

Horizontal_cast:



- не кастит через private или protected

АБСТРАКТНЫЕ КЛАССЫ И ИНТЕРФЕЙСЫ

АБСТРАКТНАЯ (ЧИСТАЯ ВИРТУАЛЬНАЯ) ФУНКЦИЯ

- `virtual type func(params) = 0;`
- ВЫНУЖДАЕТ ПОТОМКОВ ПЕРЕОПРЕДЕЛИТЬ СЕБЯ

АБСТРАКТНЫЙ КЛАСС

- КЛАСС С АБСТРАКТНОЙ ФУНКЦИЕЙ
- НЕ МОЖЕТ ИМЕТЬ ОБЪЕКТОВ

ИНТЕРФЕЙС

- МАКСИМАЛЬНАЯ АБСТРАКЦИЯ ООП
- АБСТРАКТНЫЙ КЛАСС ТОЛЬКО ИЗ АБСТРАКТНЫХ ФУНКЦИЙ В `public` (НЕТ ПОЛЕЙ)

МОДИФИКАТОРЫ `override` и `final`

`Override`

- ТИПА САМОПРОВЕРКИ
- ЕСЛИ ВИРТУАЛЬНЫЙ МЕТОД ПЕРЕОПРЕДЕЛЁН НЕВЕРНО, СРОСИТ ИСКЛЮЧЕНИЕ
- СОВЕТ: ВСЕГДА ПИСАТЬ `override`
- `virtual type func(params) override;` - СИНТАКСИС

`Final`

- ЗАПРЕЩАЕТ ПЕРЕОПРЕДЕЛЯТЬ МЕТОД/НАСЛЕДОВАТЬ КЛАСС ДАЛЕЕ
- `virtual type func(params) override final;`
- `class Name final : ... {`